

Sounds of Seattle: Classifying Seattle Bird Calls with Convolutional Neural Networks

Tyler Franck

May 22, 2025

Abstract

In this report, we explore the application of convolutional neural networks (CNN) to classify Seattle bird calls, using spectrograms as visual representations of the audio data. Undersampling was used to address class imbalance, and the data was scaled to the $[0,1]$ interval to improve model convergence. We focus on two tasks: binary classification (distinguishing between the American Crow and Red-winged Blackbird) and multi-class classification (distinguishing among 12 bird species). For each task, both lightweight (less layers and parameters) and heavyweight (more layers and parameters) models were trained and compared. The multi-class model was then used to classify three external test files for which the true labels are unknown. We find that the models for both tasks perform markedly better than naive guessing, but not enough for the classifications on the external files to be convincing. These results suggest that while CNNs are theoretically well-suited for this task, further improvements in network architecture and additional training data may be necessary to improve multi-class classification confidence.

Contents

1	Introduction	2
2	Theoretical Background	2
2.1	Neural Networks	2
2.2	Convolutional Neural Networks	3
2.3	Spectrograms	4
3	Methodology	5
3.1	Data Selection and Preprocessing	5
3.2	Model Details	6
3.2.1	Binary Classification Models	6
3.2.2	Multi-class Classification Models	7
3.3	External Test Data	9
4	Results	9
4.1	Binary Classification: American Crow vs. Red-winged Blackbird	9
4.2	Multi-class Classification	11
4.3	External Test Data	13
5	Discussion	13
5.1	Binary Classification	13
5.2	Multi-class Classification	13
5.3	External Test Data	13
6	Conclusion	14
7	References	15

12 Species of Seattle Birds



Figure 1: The 12 bird species included in the dataset.

1 Introduction

In this report, we will explore the application of convolutional neural networks (CNN) to classify Seattle bird calls. In particular, we will develop CNNs for two tasks: binary classification between the American Crow and the Red-winged Blackbird, and multi-class classification between all 12 species in the dataset. The original data was taken from the Birdcall competition data(4), which itself was taken from the crowd-sources wildlife sound archive Xeno-Canto(6). We will be using a processed version of the data(3) that was subsetting to only include high quality data for 12 species of birds (see Figure 1) and processed to compute the *spectrograms* of the audio files. A spectrogram is a visual representation of sound frequency over time. You can think of it as an *image* of a sound. Importantly, using spectrograms is what allows us to classify sounds using image classification techniques such as CNNs.

2 Theoretical Background

2.1 Neural Networks

Neural networks are a highly flexible class of models that draw inspiration from the structure of the brain. At a high level, neural networks try to mimic neural connectivity patterns and activation mechanisms using **nodes** with *activation values* and (directed) **edges** with *transmission weights*: Each node stores a numerical value that represents its activation state (like a neuron firing or not firing), and each edge stores a weight representing the strength and direction (positive or negative) of signal transmission. When a signal propagates through the network, each node's value is scaled by and transmitted along its outgoing edges to connected nodes, who sum the incoming scaled values and apply a (non-linear) **activation function**¹ to compute *their* activation state. Information starts at an **input layer** (where node values encode features of the data) and then propagates through the **hidden layers** to ultimately produce values at the **output layer** (where node values encode model predictions)². Figure 2 shows the simplest kind of neural network—the **feedforward neural network**.

¹While there are many possible choices for the activation function, it must be *non-linear*. The edge transmissions already perform linear transformations, so there is no value in tacking on a linear activation. It is the non-linearity of their activation functions that give neural networks their flexibility; if a neural network used only linear activation functions, then it would amount to little more than convoluted, overparameterized linear regression.

²In classification tasks, we apply particular activation functions to the output layer so that the output can be interpreted as class *probabilities*. In binary classification, we use the **sigmoid** activation function with a single output node. In multi-class classification, we use the **Softmax** activation function with one output node per class.

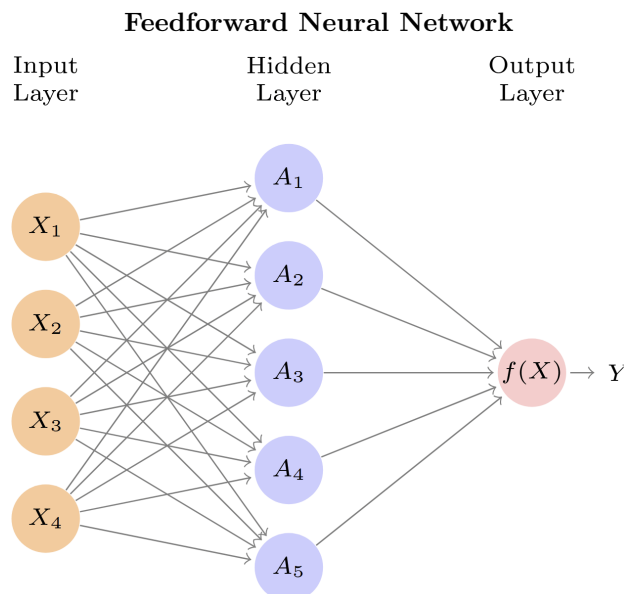


Figure 2: A simple feedforward neural network with only a single hidden layer and output node. In general, neural networks can have multiple hidden layers and multiple output nodes.

2.2 Convolutional Neural Networks

Convolutional neural networks (CNN) are a type of neural network that is especially well-suited for image processing tasks like image classification. They are designed to mimic how humans process images by recognizing compositional features/patterns in the image that distinguish different objects (e.g., tigers have *ears*, cars have *wheels*). CNNs first identify low-level features in the image (small edges, patches of color, and so on) and then combines them to form higher-level features (ears, eyes, and so on). Eventually, the presence or absence of these higher-level features contributes to a higher or lower probability of any given class.

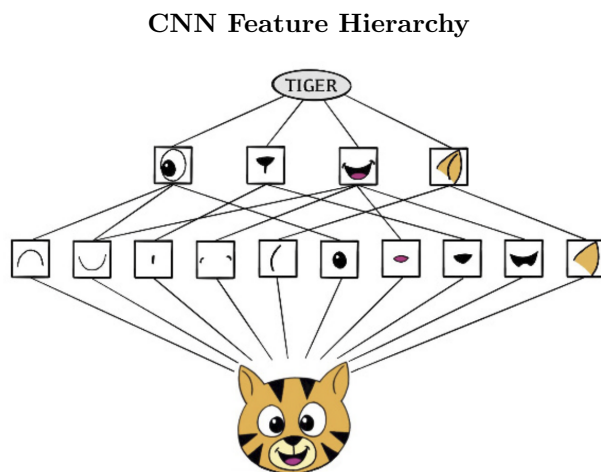


Figure 3: A diagram showing how CNNs classify images using a feature hierarchy built up from low-level features.

CNNs are able to construct this feature hierarchy by using a combination of **convolution layers** and **pooling layers**. Convolution layers consist of **convolution filters**, which function as specialized templates designed to identify a particular local feature or pattern in an image. In other words, they're *feature detectors*.

Convolution filters work by **convolving** the filter with a submatrix of the image (like a matrix dot product): if a submatrix resembles the filter, then its convolution with the filter will be large; otherwise, if the submatrix and filter are dissimilar, their convolution will be small. By sliding the filter window around the image and computing the convolutions of the region below, we obtain the **convolved image**, which highlights regions of the original image that match the convolution filter. The pooling layers then simply condense this image into a smaller summary image by calculating representative aggregates of non-overlapping chunks of the image.³

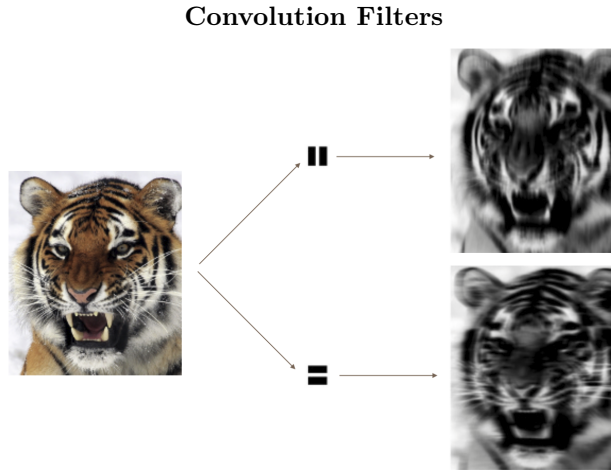


Figure 4: The convolved image highlights regions of the original image that match the convolution filter. The upper convolved image highlights the tiger’s vertical stripes whereas the bottom convolved image highlights the tiger’s horizontal stripes.

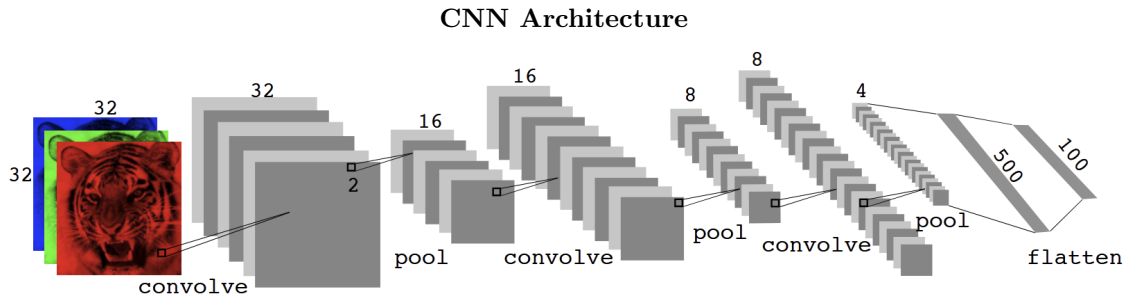


Figure 5: Example CNN architecture for an image classification task

The architecture of a CNN typically alternates between convolution layers and pooling layers. The early layers serve to capture low-level features (e.g., edges), and the deeper layers build off of these low-level features to capture high-level features (e.g., an ear). Eventually, this information is flattened and fed into fully-connected layer(s) that perform higher-level reasoning on the extracted features to produce the model prediction.

2.3 Spectrograms

Audio data describes changes in air pressure (sound) over time. While this time-series representation contains all the information, it can be difficult for models to extract meaningful features from raw waveform data.

³For example, the 2×2 max pooling operation summarizes each non-overlapping 2×2 block of pixels in an image using the maximum value in the block, which reduces the size of the image by a factor of 2 in each direction.

Spectrograms provide an alternative representation that “effectively shortcut the task of learning different filters to apply to the raw waveform.”(1)

Spectrograms are created by applying the **Fast Fourier Transform** (FFT) to (potentially overlapping) time windows, which reveals which frequencies are present at each point in time. The resulting spectrogram is usually displayed as a two-dimensional image, with time along the x-axis, frequency along the y-axis, and intensity encoded with color or brightness intensity (color or brightness). This *visual* representation of *audio* data turns an audio classification problem into an image classification problem, which is what allows us to use convolutional neural networks.

3 Methodology

3.1 Data Selection and Preprocessing

There is a moderate class imbalance (see Table 1). In particular, there are 630 samples of the House Sparrow but only 37 samples of the Northern Flicker. If we do nothing to address this, then the models will learn to bias their predictions according to the relative class frequencies.

Species	Samples
American Crow	66
American Robin	172
Berwick’s Wren	144
Black-capped Chickadee	45
Dark-eyed Junco	125
House Finch	84
House Sparrow	630
Northern Flicker	37
Red-winged Blackbird	187
Song Sparrow	263
Spotted Towhee	137
White-crowned Sparrow	91

Table 1: Number of samples for each species.

In our case, the relative frequencies aren’t useful information, and so we want to balance the classes so that the models can focus on actually learning the features that distinguish the different species. We do so using *undersampling*, where we simply don’t use all the samples of the more common birds. Specifically, we form our datasets so that the most common class appears *at most* twice as often as the least common class (see Tables 2 and 3). In addition to undersampling, we also scale the data so that values are in the interval $[0, 1]$ to improve model convergence. Finally, we split the datasets into a training and test set using a 70-30 split.

Species	Samples	Fraction
American Crow	66	0.333
Red-winged Blackbird	132	0.667

Table 2: Number of samples for each species in the binary dataset.

Species	Samples	Fraction
American Crow	66	0.081
American Robin	74	0.091
Berwick’s Wren	74	0.091
Black-capped Chickadee	45	0.055
Dark-eyed Junco	74	0.091
House Finch	74	0.091
House Sparrow	74	0.091
Northern Flicker	37	0.045
Red-winged Blackbird	74	0.091
Song Sparrow	74	0.091
Spotted Towhee	74	0.091
White-crowned Sparrow	74	0.091

Table 3: Number of samples for each species in the multi-class dataset.

3.2 Model Details

For each task, we train two models: a “lightweight” model with fewer layers/parameters and a “heavyweight” model with more layers/parameters. As the heavyweight model has more parameters but is being trained on the same amount of data, we also include more regularization in the heavyweight models. The number of training epochs and batch size were manually tuned using validation plots and based on the size of the training set (e.g., more data allows for more epochs and a larger batch size). The lightweight and heavyweight models were then compared on the basis of final model test accuracy. The model architectures are given in the following sections.

3.2.1 Binary Classification Models

The binary training set contains only 138 samples. Training the models over 30 epochs with a batch size of 4 seemed to work well.

Layer	Type
1	Input
2	Reshape
3	Conv2D (16 filters, 3×3 kernel, ReLU activation)
4	BatchNormalization
5	MaxPooling2D (2×2 pool size)
6	SpatialDropout2D (0.2 rate)
7	Flatten
8	Dense (32 units, ReLU activation)
9	BatchNormalization
10	Dropout (0.2 rate)
11	Dense (1 unit, Sigmoid activation)

Table 4: Lightweight binary model architecture. Contains only a single set of convolution and pooling layers.

Layer	Type
1	Input
2	Reshape
3	Conv2D (16 filters, 3×3 kernel, ReLU activation, 0.001 regularizer)
4	BatchNormalization
5	MaxPooling2D (2×2 pool size)
6	Conv2D (32 filters, 3×3 kernel, ReLU activation, 0.001 regularizer)
7	BatchNormalization
8	MaxPooling2D (2×2 pool size)
9	SpatialDropout2D (0.3 rate)
10	Flatten
11	Dense (64 units, ReLU activation, 0.001 regularizer)
12	BatchNormalization
13	Dense (1 unit, Sigmoid activation)

Table 5: Heavyweight binary model architecture. Contains two sets of convolution and pooling layers.

3.2.2 Multi-class Classification Models

The multi-class training set contains 569 samples. Training the models over 50 epochs with a batch size of 16 seemed to work well.

Layer	Type
1	Input
2	Reshape
3	Conv2D (32 filters, 3×3 kernel, ReLU activation)
4	BatchNormalization
5	MaxPooling2D (2×2 pool size)
6	Conv2D (64 filters, 3×3 kernel, ReLU activation)
7	BatchNormalization
8	MaxPooling2D (2×2 pool size)
9	SpatialDropout2D (0.25 rate)
10	Flatten
11	Dense (128 units, ReLU activation)
12	BatchNormalization
13	Dropout(0.25 rate)
14	Dense (12 units, Softmax activation)

Table 6: Lightweight multi-class model architecture. Contains two sets of convolution and pooling layers.

Layer	Type
1	Input
2	Reshape
3	Conv2D (32 filters, 3×3 kernel, ReLU activation, 0.001 regularizer)
4	BatchNormalization
5	MaxPooling2D (2×2 pool size)
6	Conv2D (64 filters, 3×3 kernel, ReLU activation, 0.001 regularizer)
7	BatchNormalization
8	MaxPooling2D (2×2 pool size)
9	Conv2D (128 filters, 3×3 kernel, ReLU activation, 0.001 regularizer)
10	BatchNormalization
11	MaxPooling2D (2×2 pool size)
12	SpatialDropout2D (0.25 rate)
13	Flatten
14	Dense (256 units, ReLU activation, 0.001 regularizer)
15	BatchNormalization
16	Dropout(0.25 rate)
17	Dense (12 units, Softmax activation)

Table 7: Heavyweight multi-class model architecture. Contains three sets of convolution and pooling layers.

3.3 External Test Data

To evaluate the real-world performance of our trained models, we were given three external test files (in MP3 format) for which the true labels are unknown. In order to guarantee data consistency, we compute their spectrograms using the same preprocessing methodology used for the original dataset(3). The waveforms and their corresponding spectrograms are shown below.

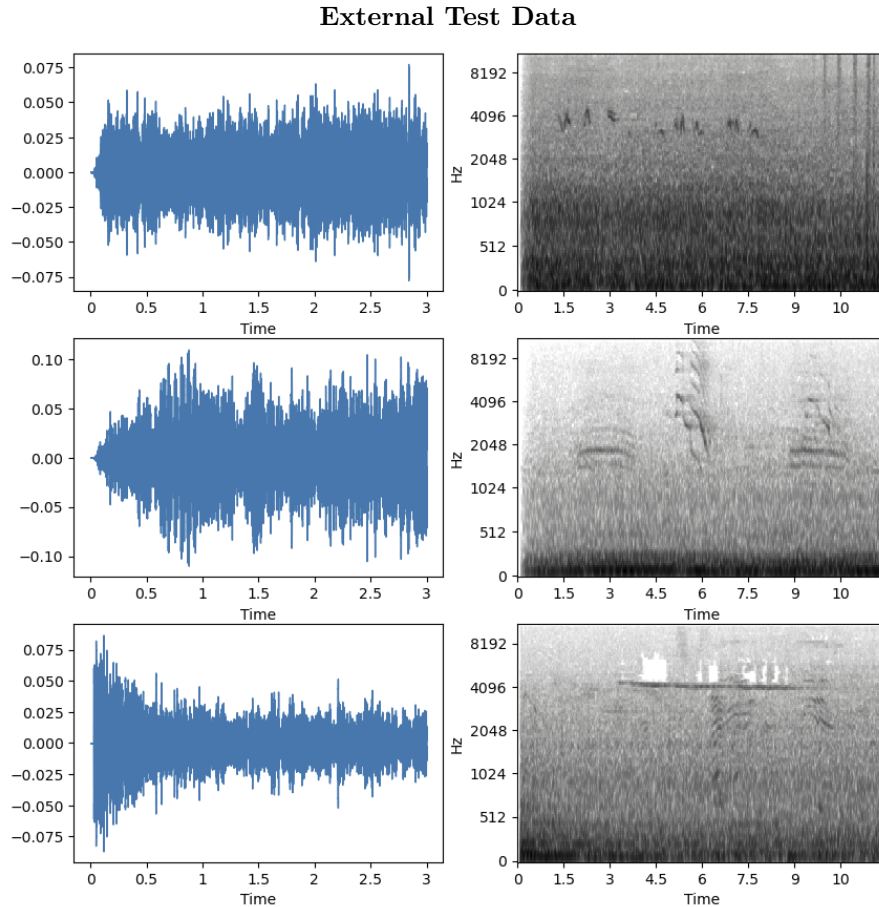


Figure 6: The waveforms (left) and spectrograms (right) of the three external test files.

4 Results

4.1 Binary Classification: American Crow vs. Red-winged Blackbird

Model	Train Time (s)	Test Accuracy
Lightweight Binary	50.7	78.3%
Heavyweight Binary	88.2	83.3%

Table 8: Binary classification model metrics.

Lightweight Validation Plot

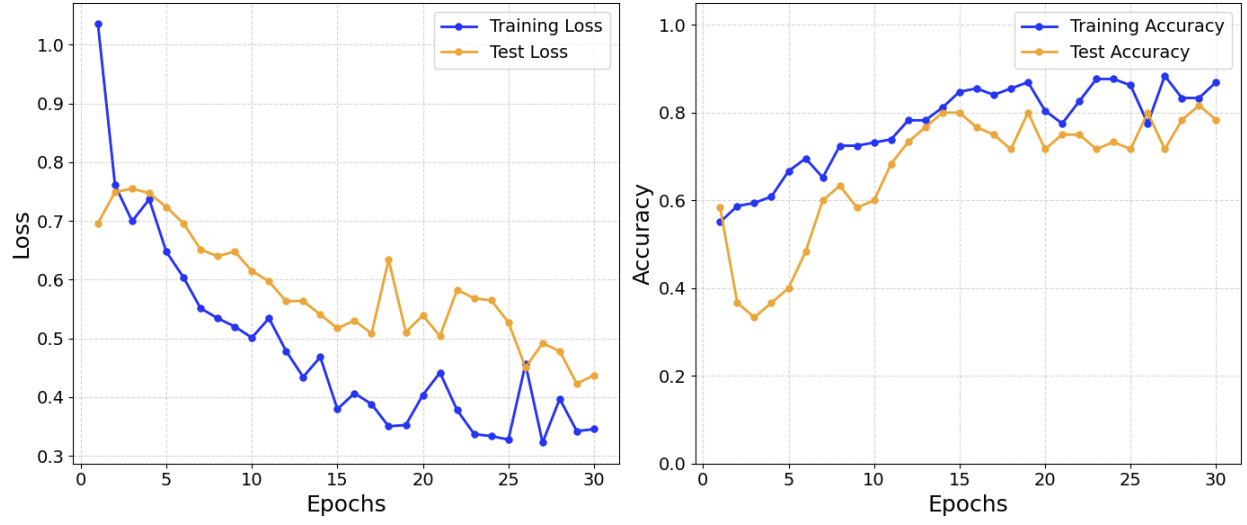


Figure 7: Training and test loss (left) and accuracy (right) by epoch.

Heavyweight Validation Plot

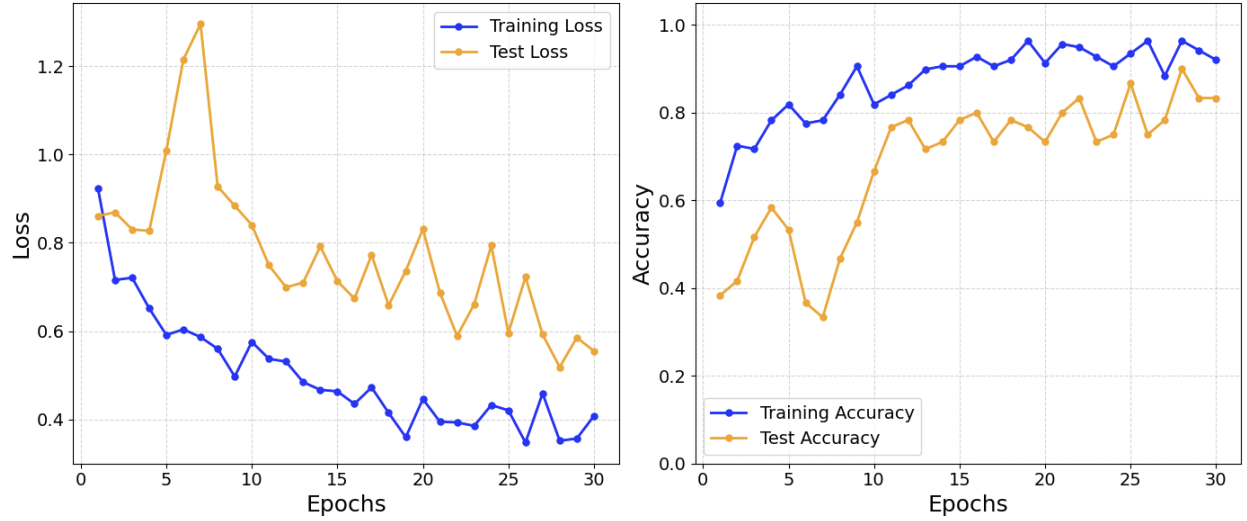


Figure 8: Training and test loss (left) and accuracy (right) by epoch.

Predicted\Truth	American Crow	Red-winged Blackbird
American Crow (L)	13	6
Red-winged Blackbird (L)	7	34
American Crow (H)	13	3
Red-winged Blackbird (H)	7	37

Table 9: Confusion matrices for the two models on the test set. The true class is given by the column, while the model and predicted class is given by the row. The lightweight model evenly distributed errors between the two classes; the heavyweight model improves on the lightweight model by correcting 3 errors.

4.2 Multi-class Classification

Model	Train Time (min)	Test Accuracy
Lightweight Multi-class	12.32	31.0%
Heavyweight Multi-class	19.34	35.5%

Table 10: Binary classification model metrics.

Lightweight Validation Plot

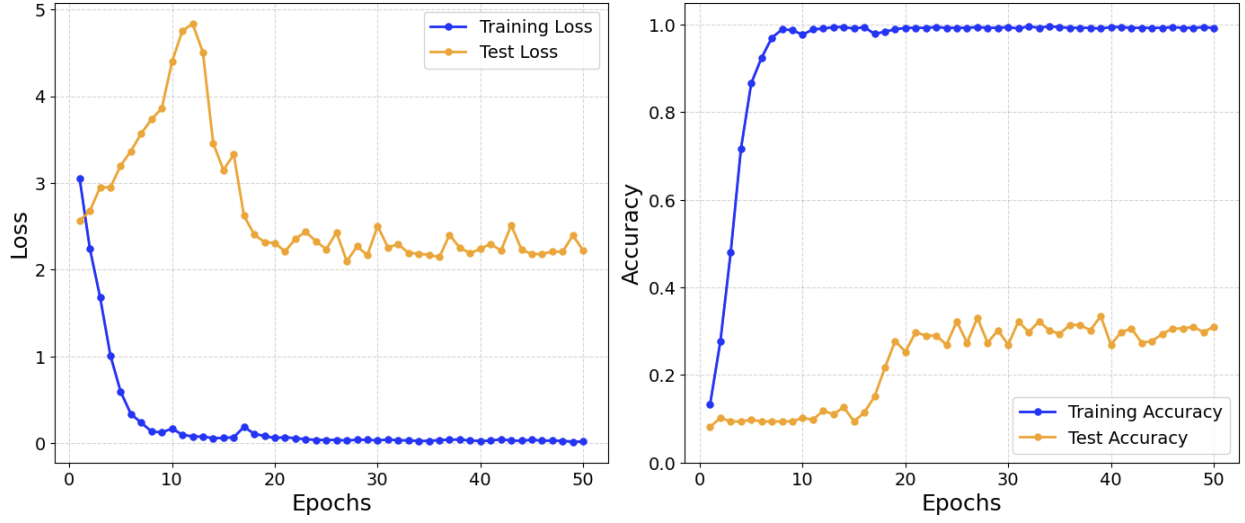


Figure 9: Training and test loss (left) and accuracy (right) by epoch.

Heavyweight Validation Plot

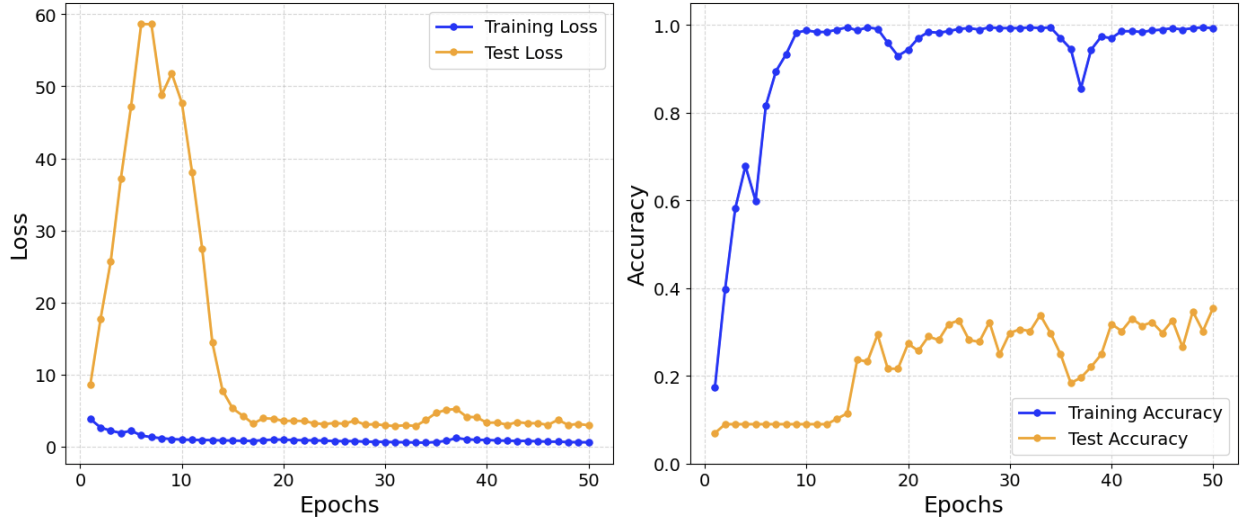


Figure 10: Training and test loss (left) and accuracy (right) by epoch.

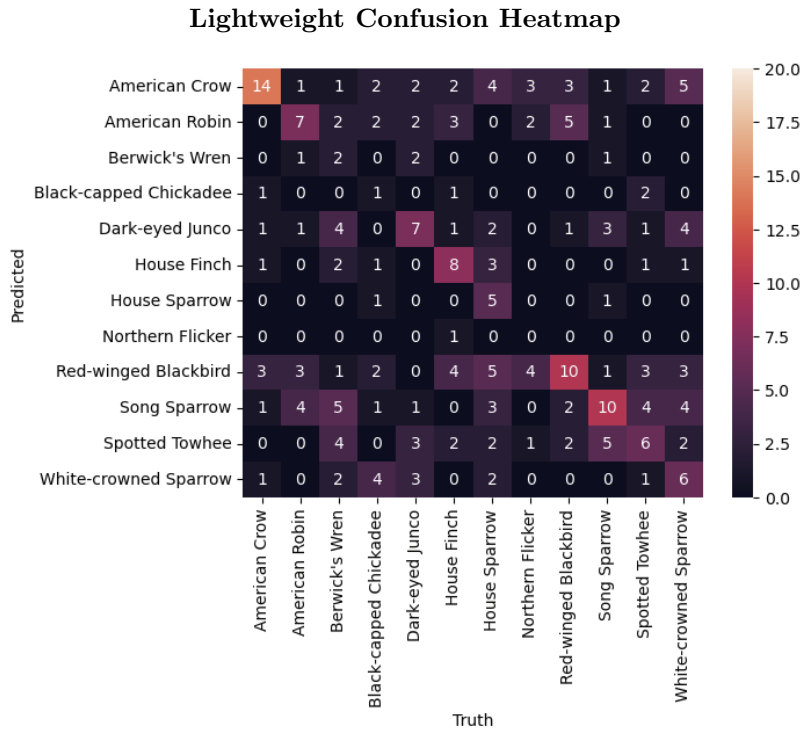


Figure 11: Heatmap of the lightweight model's confusion matrix on the test set.

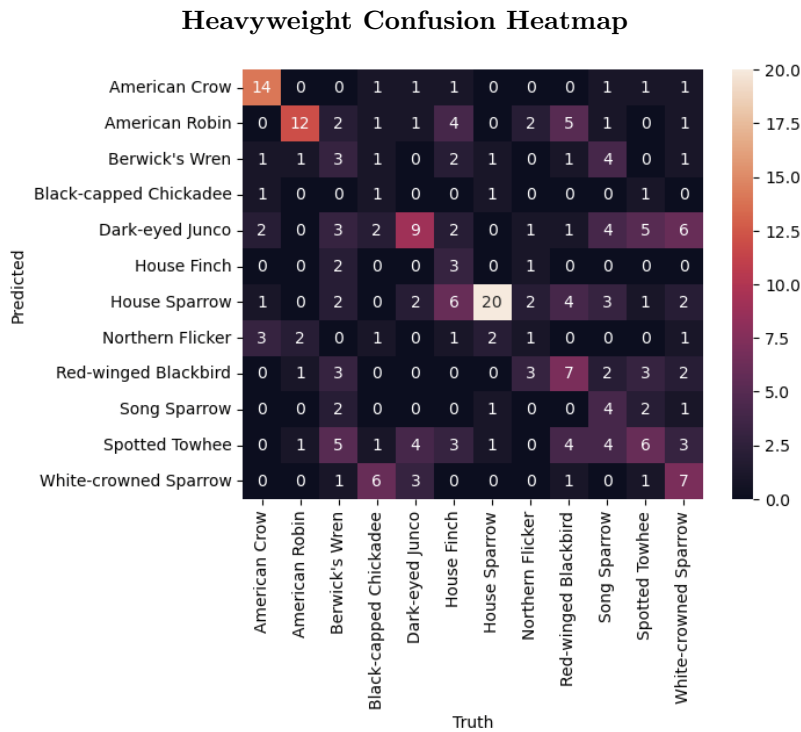


Figure 12: Heatmap of the heavyweight model's confusion matrix on the test set.

4.3 External Test Data

File	Prediction (L)	Prediction (H)
test1.mp3	Song Sparrow	House Sparrow
test2.mp3	American Robin	House Sparrow
test3.mp3	Song Sparrow	House Sparrow

Table 11: The heavyweight (H) and lightweight (L) multi-class model predictions on the external test data.

5 Discussion

5.1 Binary Classification

The lightweight model trained for 50.7 seconds and achieved a test accuracy of 78.3%, while the heavyweight model trained for 88.2 seconds and achieved a test accuracy of 83.3% (Table 8). Importantly, both models achieve a test accuracy well above 66.7%—the accuracy we’d get by always predicting the more common Red-winged Blackbird (Table 2). With 66.7% as a baseline, a 5% advantage in test accuracy is actually quite significant.

Looking at Figures 7 and 8, we see that the validation plots for the lightweight and heavyweight models show a little volatility but no overfitting. In tuning the number of training epochs, I found that the models would quickly start to overfit after around 35 epochs, which is why we stop at 30 even though it may appear like further progress can be made.

Finally, the confusion matrices (Table 9) of the two models are fairly impressive. The lightweight model evenly distributes its errors between the two classes, mistakenly classifying American Crows as Red-winged Blackbirds 7 times, and mistakenly classifying Red-winged Blackbirds and American Crows 6 times. While the heavyweight model’s confusion matrix is less balanced, it is also *strictly better* than the lightweight model, only mistaking Red-winged Blackbirds for American Crows 3 times.

5.2 Multi-class Classification

The lightweight model trained for 12.32 minutes and achieved a test accuracy of 31%, while the heavyweight model trained for 19.34 minutes and achieved a test accuracy of 35.5%. Again, it is important to consider the relative class frequencies as a baseline: by simply guessing (one of) the most common classes, we’d get around 9.1% accuracy (Table 3). So, while 35.5% accuracy doesn’t sound that impressive, it is significantly better than *chance*.

Looking at the validation plots (Figures 9 and 10), we see that the metrics improved until around 20 epochs, after which it plateaued. The fact that the models neither improved nor got worse (i.e., overfitting) is quite puzzling. One possible explanation is that the data is much more complex than the models flexible (making overfitting impossible), however this explanation seems unlikely considering the fact that we used neural networks with many parameters.

Given that the multi-class models distinguish between 12 classes instead of 2, the confusion matrices for the multi-class models (Figures 11 and 12) are much more complex than those of the binary models. It’s hard to say anything general, but it can be interesting to see which birds the models had the most trouble with. Both the lightweight and heavyweight models were particularly bad at identifying the Northern Flicker, with the lightweight model failing to correctly identify it a single time and the heavyweight model only getting it right once. Instead, the models seemed to confuse the Northern Flicker with the Red-winged Blackbird, which itself is confusing because the two birds sound nothing alike! Perhaps this is a consequence of using spectrograms. More likely, it’s probably a consequence of limited training data: the Northern Flicker only had 37 samples in the entire dataset.

5.3 External Test Data

The heavyweight multi-class model classified all three test files as House Sparrow, while the lightweight multi-class model classified the first and third test files as Song Sparrow and the second test file as Amer-

ican Robin (Table 11). On the one hand, it’s encouraging that both the models agree that the bird is some kind of Sparrow—Song Sparrows and House Sparrows sound similar, so the predictions are actually somewhat consistent. On the other hand, it’s suspicious that the models think all three test files are Sparrows. Furthermore, our models only achieved around 30% test accuracy. How confident can we be in these predictions?

If you look at the waveforms/spectrograms (Figure 6) and listen to the test files, it becomes apparent that the test files are, well, *noisy*. In particular, some of the test files have background noise and even *multiple* bird calls! Given all of this, I wouldn’t be too confident in our predictions.

6 Conclusion

In this project, we built binary and multi-class classification CNN models to classify Seattle bird calls using spectrogram representations. The binary classification models performed quite well, with the heavyweight model achieving 83.3% test accuracy in distinguishing American Crows and Red-winged Blackbirds. The multi-class classification task, where we had to distinguish between all 12 bird species, proved much more challenging. While both models achieved test accuracies well above random chance—31% 9.1% and 35.5% vs. 9.1% for the lightweight and heavyweight models, respectively—the accuracy isn’t high enough for us to be *confident* in their predictions. Some possible explanations for this lackluster performance are (1) either the tested models just don’t stack up to the task (i.e., the models are poor and/or the data is simply hard to separate), or (2) the training set is not representative of the test set.

While *possible* with the number of samples in the dataset, the latter explanation is *unlikely*. The first explanation has a lot more potential, especially since it would explain the interesting plateau effect seen in Figure 10. This raises the question, are CNNs really the best model for the task? As the task involves *non-linear* classification, some other candidate models are decision trees, support vector machines, and K-nearest neighbors. Unfortunately, the data is very high-dimensional (the flattened spectrograms are length 66176) but with a low number of samples, and so these methods would likely require dimensionality reduction and/or feature engineering. One of the advantages of neural networks is that they automatically learn features from the data, which reduces the need for manual feature engineering. Furthermore, CNNs are literally designed *for* computer vision tasks, and have very useful properties such as *local connectivity* and *translation invariance*(5). This is all to say that CNNs are surely the best model for the task; getting better model performance is just a matter of finding the right architectural configuration and collecting more training data.

7 References

- [1] From sound to images, part 1: A deep dive on spectrogram creation. Macaulay Library. (2021, July 19). <https://www.macaulaylibrary.org/2021/07/19/from-sound-to-images-part-1-a-deep-dive-on-spectrogram-creation/>
- [2] Karpathy, A. (n.d.). A recipe for training neural networks. <https://karpathy.github.io/2019/04/25/recipe/>
- [3] Mendible, A. (n.d.). DATA 5322 Homework 3. <https://github.com/mendible/5322/tree/main/Homework%203>
- [4] Vopani. (2020, September 14). Xeno-Canto Bird Recordings Extended (A-M). Kaggle. <https://www.kaggle.com/datasets/rohanrao/xeno-canto-bird-recordings-extended-a-m>
- [5] Wikimedia Foundation. (2025, May 21). Convolutional Neural Network. Wikipedia. https://en.wikipedia.org/wiki/Convolutional_neural_network
- [6] Xeno-Canto. (n.d.). <https://xeno-canto.org/>